

阿里高级前端面经

一面（技术基础） · 1-10

1. 请简述CSS 盒模型以及 BFC 的触发条件和应用场景。
2. 谈谈你对原型链的理解，以及如何实现一个深拷贝。
3. 说一下事件循环（Event Loop）中宏任务和微任务的执行顺序。
4. Promise 有哪几种状态？如何实现 Promise.all？
5. 什么是闭包？它在实际开发中有哪些应用场景？
6. 手写代码：实现一个防抖函数（debounce）。
7. 谈谈 HTTP 缓存机制，强缓存和协商缓存的区别。
8. TypeScript 中 interface 和 type 的区别是什么？
9. 数组中有哪些常用的高阶函数？如何实现数组扁平化？
10. 算法题：给定一个字符串，找出其中不含有重复字符的最长子串的长度。

二面（深入原理与项目） · 1-12

1. React 的 Fiber 架构解决了什么问题？它的工作原理是什么？
2. 谈谈 Webpack 的构建流程，以及如何优化打包速度。
3. 你们项目里是如何进行性能优化的？有哪些量化指标？
4. 详细说明 Vue 或 React 的 Diff 算法差异。
5. 什么是微前端？你们在项目中是如何落地微前端架构的？
6. 请解释一下 HTTPS 的握手过程。
7. Node.js 的多进程模型是怎麼样的？如何实现进程间通信？
8. 谈谈你对前端工程化的理解，CI/CD 流程是怎么设计的？
9. 如何处理前端的大量数据渲染问题？
10. 介绍一个你遇到过的最难的技术问题，你是如何解决的？
11. 手写代码：实现一个简单的 EventEmitter（发布订阅模式）。
12. 谈谈 CSS Modules 和 CSS-in-JS 的优缺点。

三面（架构与综合能力） 1-8

1. 如果让你从零设计一个前端监控系统，你会考虑哪些模块？
2. 在大型项目中，如何进行技术选型？

3. 聊聊你对“前端智能化”或“低代码”的看法。
4. 你们团队是如何保证代码质量和项目稳定性的？
5. 谈谈你在过去一年中技术上的最大成长。
6. 如何看待前端框架的频繁更迭？你如何保持自己的竞争力？
7. 设计一个组件库时，你会考虑哪些因素？
8. 假如项目进度严重滞后，作为技术负责人你会怎么办？

一面（技术基础）

1. 请简述 CSS 盒模型以及 BFC 的触发条件和应用场景。

难度：★

考点：CSS 布局基础

答案要点：

盒模型决定了元素如何在页面上占据空间，而 BFC 是页面上的一个独立渲染区域。

- 盒模型：分为标准盒模型（content-box）和怪异盒模型（border-box），区别在于 width 是否包含 padding 和 border。
- BFC 触发：可以通过 overflow: hidden、display: flex/inline-block、position: absolute/fixed 等方式触发。
- 应用场景：解决外边距重叠（Margin Collapse）、清除浮动、防止元素被浮动元素覆盖。
- 渲染规则：BFC 内部的元素在垂直方向一个接一个排列，计算高度时浮动元素也参与计算。

常见坑：

- 混淆标准盒模型和 IE 盒模型的计算公式。
- 认为设置 display: block 就能触发 BFC。

追问：

- 如果想让两个相邻 div 的 margin 不重叠，除了 BFC 还有什么办法？

2. 谈谈你对原型链的理解，以及如何实现一个深拷贝。

难度：★★

考点：JS 核心原理

答案要点：

原型链是 JS 实现继承的基础，通过 **proto** 指针将对象连接起来；深拷贝则是为了彻底断开引用。

- 原型链：每个对象都有原型，查找属性时会顺着原型链向上直到 null。
- 深拷贝实现：简单场景用 JSON.parse，复杂场景需递归处理。
- 循环引用：使用 WeakMap 存储已拷贝对象，防止死循环。

- 特殊类型：需处理 Date、RegExp、Map、Set 等特殊对象的克隆。

常见坑：

- 忽略 Symbol 属性的拷贝。
- 使用 typeof 判断 null 时返回 object 的陷阱。

追问：

- 为什么深拷贝中推荐使用 WeakMap 而不是 Map？

3. 说一下事件循环（Event Loop）中宏任务和微任务的执行顺序。

难度：★★

考点：异步执行机制

答案要点：

事件循环是 JS 处理异步任务的核心机制，遵循先执行同步代码，再清空微任务队列，最后执行宏任务的顺序。

- 执行流程：同步代码 -> 微任务（Promise.then, MutationObserver） -> 渲染 -> 宏任务（setTimeout, I/O）。
- 嵌套情况：宏任务执行过程中产生的微任务会在当前宏任务结束后立即执行。
- Node.js 差异：Node 11 之后与浏览器行为趋同，但仍有 process.nextTick 等特殊阶段。
- 渲染时机：浏览器通常在一次宏任务执行完且微任务清空后尝试进行 UI 渲染。

常见坑：

- 错误认为 Promise 构造函数内部是异步的。
- 混淆 await 后面的代码执行时机。

追问：

- 如果在微任务里不断递归产生新的微任务，页面会发生什么？

4. Promise 有哪几种状态？如何实现 Promise.all？

难度：★★

考点：异步编程

答案要点：

Promise 解决了回调地狱问题，具有 Pending、Fulfilled、Rejected 三种不可逆的状态。

- Promise.all 特点：所有 Promise 都成功才成功，只要有一个失败就立即失败。
- 实现逻辑：返回一个新的 Promise，内部维护一个计数器和结果数组。
- 结果顺序：通过索引确保输出数组的顺序与输入 Promise 数组顺序一致。
- 边界处理：处理输入参数不是数组或包含非 Promise 值的情况。

常见坑：

- 忘记处理 Promise.all 的空数组情况。

- 没考虑到输入数组中普通值的转换。

追问：

- Promise.allSettled 和 Promise.all 的区别是什么？

5. 什么是闭包？它在实际开发中有哪些应用场景？

难度：★

考点：作用域

答案要点：

闭包是指有权访问另一个函数作用域中变量的函数，本质是函数与其词法环境的引用捆绑。

- 形成条件：函数嵌套且内部函数引用了外部函数的变量。
- 应用场景：封装私有变量、柯里化（Currying）、防抖节流、模块化模式。
- 内存影响：闭包会导致外部函数的变量常驻内存，不当使用可能引起内存泄漏。
- 作用域链：闭包通过保存对外部词法环境的引用来维持变量的可访问性。

常见坑：

- 在循环中使用 var 定义变量导致闭包引用同一个值。
- 忽略手动解除闭包引用导致内存无法回收。

追问：

- 现代浏览器引擎对闭包的内存优化做了哪些工作？

6. 手写代码：实现一个防抖函数（debounce）。

难度：★

考点：手写代码基础

答案要点：

防抖是在事件触发后延迟执行，如果在延迟时间内再次触发，则重新计时。

- 核心逻辑：使用 setTimeout 延迟执行，并在每次触发时 clearTimeout。
- this 指向：确保执行函数时的 this 指向正确。
- 参数传递：支持透传事件对象等参数。
- 立即执行：可选支持第一次触发是否立即执行。

代码块

```
1 function debounce(fn, wait) {
2   let timer = null;
3   return function(...args) {
4     if (timer) clearTimeout(timer);
5     timer = setTimeout(() => {
6       fn.apply(this, args);
7     }, wait);
  }
```

```
8    };  
9    }
```

常见坑：

- 忘记清除定时器。
- 箭头函数导致 this 绑定失效。

7. 谈谈 HTTP 缓存机制，强缓存和协商缓存的区别。

难度：★★

考点：网络协议

答案要点：

HTTP 缓存通过减少重复请求来提升性能，分为浏览器本地判断的强缓存和需服务器确认的协商缓存。

- 强缓存：使用 Cache-Control (max-age) 或 Expires，命中时不发送请求。
- 协商缓存：使用 ETag/If-None-Match 或 Last-Modified/If-Modified-Since。
- 优先级：Cache-Control 优先级高于 Expires，ETag 优先级高于 Last-Modified。
- 状态码：强缓存返回 200 (from cache)，协商缓存命中返回 304。

常见坑：

- 认为 304 也会传输完整的响应体。
- 忽略了 Ctrl+F5 强制刷新对缓存的影响。

追问：

- 为什么 ETag 比 Last-Modified 更可靠？

8. TypeScript 中 interface 和 type 的区别是什么？

难度：★★

考点：TS 基础

答案要点：

两者都用于定义类型，但在扩展能力和适用场景上有所不同。

- 扩展方式：interface 通过 extends 扩展，type 通过 & 交叉类型扩展。
- 合并声明：interface 支持同名自动合并，type 不支持。
- 适用范围：type 可以定义基本类型别名、联合类型、元组，interface 主要用于定义对象结构。
- 编译性能：在复杂类型计算中，interface 的表现通常优于 type。

常见坑：

- 尝试在 type 中使用 implements。
- 混淆声明合并和覆盖。

追问：

- 什么是泛型约束 (Generic Constraints) ?

9. 数组中有哪些常用的高阶函数？如何实现数组扁平化？

难度：★

考点：JS API

答案要点：

高阶函数是接收函数作为参数或返回函数的函数，常用于集合处理。

- 常用函数：map, filter, reduce, some, every, find。
- 扁平化方法：Array.prototype.flat()、递归处理、扩展运算符结合 some。
- reduce 实现：通过 reduce 累加器递归拼接数组。
- 性能考量：对于超大数组，递归可能导致栈溢出，需考虑迭代解法。

常见坑：

- map 忘记写 return 导致结果全是 undefined。
- reduce 没传初始值导致空数组报错。

追问：

- reduce 如何实现 map 的功能？

10. 算法题：给定一个字符串，找出其中不含有重复字符的最长子串的长度。

难度：★★

考点：滑动窗口

答案要点：

使用滑动窗口技术可以高效解决此类子串问题。

- 核心思路：维护一个左指针和右指针，利用 Map 记录字符出现的位置。
- 窗口移动：右指针向右移动，若遇到重复字符，左指针跳到重复字符上次出现位置的下一位。
- 结果更新：每次移动右指针后，更新最大长度。
- 时间复杂度： $O(n)$ ，空间复杂度 $O(\min(m, n))$ 。

代码块

```
1  function lengthOfLongestSubstring(s) {
2      let map = new Map(), max = 0, left = 0;
3      for (let right = 0; right < s.length; right++) {
4          if (map.has(s[right])) {
5              left = Math.max(map.get(s[right]) + 1, left);
6          }
7          map.set(s[right], right);
8          max = Math.max(max, right - left + 1);
9      }
```



```
10     return max;
11 }
```

常见坑：

- 左指针回退问题（必须取 Math.max 确保左指针不向左跳）。

二面（深入原理与项目）

1. React 的 Fiber 架构解决了什么问题？它的工作原理是什么？

难度：★★★

考点：React 原理

答案要点：

Fiber 引入了可中断的异步渲染机制，解决了 React 15 在处理大量 DOM 节点时主线程卡顿的问题。

- 核心矛盾：JS 执行时间过长导致浏览器无法及时响应高优先级任务（如动画、输入）。
- 协作式调度：将渲染任务拆分为小片（Fiber），利用 requestIdleCallback 机制在浏览器空闲时执行。
- 双缓存技术：维护 current 树和 workInProgress 树，在内存中构建好后再统一提交。
- 阶段划分：分为 Reconcile（可中断）和 Commit（不可中断）两个阶段。

常见坑：

- 认为 Fiber 提高了代码执行的绝对速度（实际上是提高了响应性）。
- 混淆了 Fiber 节点和真实 DOM 节点的关系。

追问：

- 为什么 React 不直接使用 Generator 来实现中断？

2. 谈谈 Webpack 的构建流程，以及如何优化打包速度。

难度：★★

考点：工程化

答案要点：

Webpack 是一个串行执行的构建流程，从入口开始递归解析依赖并调用 Loader 和 Plugin。

- 核心流程：初始化参数 -> 开始编译 -> 确认入口 -> 编译模块（Loader） -> 完成模块编译 -> 输出资源（Chunk） -> 写入文件。
- 速度优化：使用持久化缓存（cache: filesystem）、多进程打包（thread-loader）、缩小搜索范围（include/exclude）。
- 体积优化：Tree Shaking、代码分割（SplitChunks）、压缩图片、CDN 引入外部库。

- 调试工具：使用 speed-measure-webpack-plugin 分析耗时，使用 webpack-bundle-analyzer 分析体积。

常见坑：

- Loader 配置顺序写反（从右往左执行）。
- 在开发环境下开启了生产环境才需要的耗时优化。

追问：

- Loader 和 Plugin 的本质区别是什么？

3. 你们项目里是如何进行性能优化的？有哪些量化指标？

难度：★★★

考点：性能优化实战

答案要点：

性能优化应遵循“指标采集-瓶颈分析-针对优化-持续监控”的闭环。

- 核心指标：LCP（最大内容绘制）、FID（首次输入延迟）、CLS（累计布局偏移）、FCP。
- 网络层面：HTTP/2、Gzip/Brotli 压缩、预加载（preload/prefetch）、图片懒加载。
- 渲染层面：减少重排重绘、防抖节流、虚拟列表（Virtual List）、SSR/SSG。
- 代码层面：按需加载、组件懒加载、避免大型依赖包。

常见坑：

- 盲目优化没有数据支撑的环节。
- 忽略了移动端弱网环境下的表现。

追问：

- 如果页面出现长列表卡顿，除了虚拟列表还有什么方案？

4. 详细说明 Vue 或 React 的 Diff 算法差异。

难度：★★★

考点：框架对比

答案要点：

Diff 算法是为了最小化 DOM 操作，两者都采用了同层比较的策略，但在具体实现上有差异。

- React：采用单向遍历，对比新旧集合，通过 key 发现可复用节点，主要移动旧节点。
- Vue2：双端比较（双指针），同时从两端向中间遍历，效率较高。
- Vue3：借鉴了 ivi 和 inferno，采用最长递增子序列算法来减少不必要的移动。
- 共同点：只做同级比较，不同类型组件直接销毁重建。

常见坑：

- 认为 key 使用 index 没问题（会导致错误的复用和性能下降）。

- 混淆了虚拟 DOM 和真实 DOM 的更新时机。

追问：

- 为什么 React 不采用双端 Diff？

5. 什么是微前端？你们在项目中是如何落地微前端架构的？

难度：★★★

考点：架构设计

答案要点：

微前端将大型应用拆分为独立运行、独立开发、独立部署的微应用，解决巨石应用维护难的问题。

- 核心挑战：应用加载、样式隔离、状态共享、路由同步。
- 隔离方案：CSS 隔离（Shadow DOM, Scoped CSS）、JS 隔离（Proxy Sandbox, Snapshot Sandbox）。
- 通信机制：CustomEvent、发布订阅模式、全局状态管理。
- 常用框架：qiankun（基于 single-spa）、wujie（基于 iframe）。

常见坑：

- 忽略了子应用卸载时的内存清理。
- 多个子应用共用全局第三方库导致的冲突。

追问：

- iframe 做微前端有哪些优缺点？

6. 请解释一下 HTTPS 的握手过程。

难度：★★

考点：网络安全

答案要点：

HTTPS 在 TCP 之上增加了 SSL/TLS 层，通过对称加密和非对称加密结合保证传输安全。

- 证书校验：客户端验证服务端证书的合法性（CA 签名）。
- 密钥交换：通过非对称加密（RSA 或 Diffie-Hellman）协商出后续通信用的对称密钥。
- 完整性保护：使用 MAC 算法防止数据被篡改。
- 握手步骤：Client Hello -> Server Hello -> Certificate -> Key Exchange -> Finished。

常见坑：

- 认为 HTTPS 能够完全防止中间人攻击（如果客户端信任了伪造的根证书则不行）。
- 混淆对称加密和非对称加密的应用场景。

追问：

- 什么是数字签名？

7. Node.js 的多进程模型是怎么样？如何实现进程间通信？

难度：★★

考点：Node.js

答案要点：

Node.js 是单线程运行的，但可以通过 cluster 模块或 child_process 利用多核 CPU。

- Master-Worker：主进程负责调度，工作进程负责处理业务逻辑。
- 负载均衡：主进程监听端口，通过 Round-robin 算法分发请求给子进程。
- IPC 通信：通过管道（Pipe）实现，底层使用 libuv，通过 send 方法发送消息。
- 容错机制：监听子进程 exit 事件，实现自动重启（守护进程）。

常见坑：

- 在多个子进程中重复监听同一个端口导致报错（需了解 cluster 的句柄传递）。
- 忽略了子进程异常退出导致的僵尸进程。

追问：

- Node.js 中的 Cluster 模块是如何实现端口共享的？

8. 谈谈你对前端工程化的理解，CI/CD 流程是怎么设计的？

难度：★★

考点：工程化

答案要点：

工程化是为了提高开发效率、保证代码质量，涵盖了从开发到部署的全生命周期。

- 规范化：ESLint, Prettier, Commitlint, Git Hooks。
- 自动化：自动化测试（Jest）、自动构建、自动部署。
- CI 流程：代码提交 -> 静态检查 -> 单元测试 -> 构建制品。
- CD 流程：制品上传 -> 灰度发布 -> 全量发布 -> 监控回滚。

常见坑：

- 缺乏自动化测试，导致 CI 流程流于形式。
- 构建环境与生产环境不一致。

追问：

- 如何实现前端项目的增量发布？

9. 如何处理前端的大量数据渲染问题？

难度：★★

考点：性能优化

答案要点：

处理大数据量渲染的核心思路是“按需渲染”和“分片处理”。

- 虚拟列表：只渲染可视区域内的 DOM 节点，通过滚动偏移量动态计算。
- 时间分片：利用 requestAnimationFrame 或 setTimeout 将长任务拆分为多个小任务，避免阻塞主线程。
- Web Worker：将复杂的计算逻辑移出主线程，防止 UI 响应卡顿。
- Canvas 渲染：对于极端的图表或地图场景，使用 Canvas 代替 DOM 渲染。

常见坑：

- 虚拟列表滚动过快时的白屏问题。
- 分片处理时没有处理好任务的优先级。

追问：

- 虚拟列表如果每一项高度不固定该如何实现？

10. 介绍一个你遇到过的最难的技术问题，你是如何解决的？

难度：★★★★

考点：问题解决能力

答案要点：

该问题旨在考察候选人的排查思路、技术深度以及复盘总结能力。

- 问题背景：清晰描述问题发生的场景、影响范围。
- 排查过程：使用了哪些工具（Chrome DevTools, Charles, 内存快照等），提出了哪些假设。
- 解决方案：对比了哪几种方案，为什么最终选择了这一种。
- 沉淀思考：事后如何预防类似问题再次发生，是否沉淀了文档或工具。

常见坑：

- 描述过于业务化，缺乏技术细节。
- 解决过程全靠“试”，没有体现逻辑推导。

11. 手写代码：实现一个简单的 EventEmitter（发布订阅模式）。

难度：★★

考点：设计模式

答案要点：

发布订阅模式是前端最常用的设计模式之一，用于组件间解耦通信。

- 核心方法：on (订阅), emit (发布), off (取消订阅), once (只执行一次)。
- 数据结构：使用对象或 Map 存储事件名与回调函数数组的映射。
- once 实现：包装回调函数，在执行后立即调用 off。
- 健壮性：处理不存在的事件或重复订阅的情况。

```

1 代码块 class EventEmitter {
2      constructor() {
3          this.events = {};
4      }
5      on(name, fn) {
6          if (!this.events[name]) this.events[name] = [];
7          this.events[name].push(fn);
8      }
9      emit(name, ...args) {
10         if (this.events[name]) {
11             this.events[name].forEach(fn => fn(...args));
12         }
13     }
14 }

```

常见坑：

- off 时没有正确解绑匿名函数。
- 忘记处理 once 的参数传递。

12. 谈谈 CSS Modules 和 CSS-in-JS 的优缺点。

难度：★★

考点：样式方案

答案要点：

两者都是为了解决 CSS 全局污染问题，但在实现机制和运行时开销上有差异。

- CSS Modules：在编译阶段将类名哈希化，无运行时开销，支持成熟。
- CSS-in-JS：如 styled-components，支持动态样式，逻辑更强，但有一定运行时性能损耗。
- 作用域隔离：两者都能有效实现组件级别的样式私有化。
- 选型建议：大型性能敏感项目倾向 CSS Modules，追求开发体验和动态性的项目选 CSS-in-JS。

常见坑：

- CSS-in-JS 在高频渲染下的性能瓶颈。
- 忽略了样式提取（Extract）对首屏加载的影响。

追问：

- Tailwind CSS 为什么最近很火？它解决了什么问题？

三面（架构与综合能力）

1. 如果让你从零设计一个前端监控系统，你会考虑哪些模块？

难度：★★★

考点：系统设计

答案要点：

监控系统应覆盖错误监控、性能监控、行为监控三大维度。

- 数据采集：SDK 自动捕获（onerror, unhandledrejection）、性能 API（PerformanceObserver）、埋点上报。
- 数据传输：考虑上报频率（节流）、上报方式（Beacon API 防止页面关闭丢失）、压缩数据。
- 数据处理：清洗、聚合、错误堆栈解析（SourceMap）。
- 报警可视化：实时看板、阈值报警（钉钉/邮件）、多维分析。

常见坑：

- 监控 SDK 自身报错导致业务页面崩溃。
- 海量数据上报对服务器造成的压力。

追问：

- 如何保证监控数据的准确性和不漏报？

2. 在大型项目中，如何进行技术选型？

难度：★★★

考点：技术前瞻性与决策

答案要点：

技术选型不是选“最好的”，而是选“最合适的”。

- 维度考量：社区活跃度、团队上手成本、生态完善程度、长期维护性。
- 业务匹配：是追求极致性能（Wasm/Rust）还是追求交付效率（低代码/模板）。
- 风险评估：是否有重大 Bug、是否容易被供应商锁定、是否有平滑迁移方案。
- 选型流程：调研 -> POC 验证 -> 团队评审 -> 小规模试点 -> 全量推广。

常见坑：

- 盲目追求新技术（Hype Driven Development）。
- 忽略了现有基建的兼容性。

3. 聊聊你对“前端智能化”或“低代码”的看法。

难度：★★★

考点：行业视野

答案要点：

这些技术旨在提升生产力，将前端从重复性劳动中解放出来。

- 低代码：核心是模型驱动和组件标准化，适用于中后台、营销页等高度标准化的场景。
- 智能化：如 D2C（Design to Code），通过 AI 识别设计稿生成代码，减少 UI 还原工作。

- 价值点：降低开发门槛、缩短交付周期、统一视觉规范。
- 局限性：复杂逻辑难以配置、生成的代码可维护性差、黑盒逻辑调试难。

常见坑：

- 认为低代码可以取代所有前端开发。
- 忽略了低代码平台的扩展性设计。

4. 你们团队是如何保证代码质量和项目稳定性的？

难度：★★

考点：团队协作与流程

答案要点：

稳定性保障是一个系统工程，涉及开发、测试、发布各阶段。

- 流程保障：严格的 Code Review 制度、单元测试覆盖率要求。
- 技术保障：静态类型检查（TS）、自动化回归测试（Cypress/Playwright）。
- 灰度策略：分批次发布、A/B Testing、快速回滚机制。
- 监控预警：全链路监控、线上异常实时告警。

常见坑：

- CR 变成走过场，没有发现实质问题。
- 测试用例维护成本过高导致被放弃。

5. 谈谈你在过去一年中技术上的最大成长。

难度：★★

考点：自我驱动

答案要点：

通过具体的案例展示自己的学习路径和深度。

- 学习目标：为什么选择学习这个技术点？
- 实践过程：是在项目中落地了，还是写了开源插件，或者沉淀了技术博客？
- 深度体现：是否深入到了源码层面，或者解决了行业内的通用痛点。
- 结果产出：给团队带来了什么价值（提效、降本、避坑）。

常见坑：

- 罗列了一堆名词但没有深度。
- 学习内容与实际工作完全脱节。

6. 如何看待前端框架的频繁更迭？你如何保持自己的竞争力？

难度：★★

考点：职业态度

答案要点：

框架只是工具，核心是解决问题的能力 and 底层基础。

- 变与不变：框架在变，但 JS 基础、网络协议、浏览器原理、算法思想是不变的。
- 学习策略：掌握核心原理，做到一通百通；关注社区趋势，但不盲从。
- 竞争力构建：建立自己的知识体系，培养架构思维，提升跨端/跨栈的综合能力。
- 解决价值：技术最终为业务服务，能解决复杂业务问题的能力是核心竞争力。

常见坑：

- 陷入“学不动了”的消极情绪。
- 只会用 API，不了解底层实现。

7. 设计一个组件库时，你会考虑哪些因素？

难度：★★★

考点：架构设计

答案要点：

组件库的质量直接影响研发效率和视觉一致性。

- 设计规范：原子化设计、主题定制能力（CSS Variables）。
- 组件质量：无障碍支持（ARIA）、完备的单元测试、按需加载。
- 文档建设：交互式 Demo、清晰的 API 文档、版本变更记录。
- 工程化：自动化构建、多环境支持（CJS/ESM）、自动化发布流水线。

常见坑：

- 组件间耦合过重，导致体积臃肿。
- 缺乏破坏性变更的迁移指南。

8. 假如项目进度严重滞后，作为技术负责人你会怎么办？

难度：★★★

考点：项目管理与抗压

答案要点：

需要从透明沟通、资源协调和范围管理三个方面入手。

- 现状分析：找出滞后的根本原因（需求变更、技术难点、人力不足）。
- 范围裁剪：与产品协商，优先保住核心功能（MVP），非核心功能延后。
- 资源优化：协调内部资源支援，或者通过技术手段（如使用成熟方案）提效。
- 向上管理：及时同步风险，给出明确的解决方案和新的排期计划，切忌隐瞒。

常见坑：

- 盲目要求团队加班，导致效率反而下降。

- 隐瞒进度直到最后时刻才爆发。